This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

Illuminating Heuristic Design: Language-Model-Driven Quality-Diversity Evolution for Combinatorial Optimization

Shikun Chen and Yangguang Liu

***Abstract*—Current methods that use large language models to evolve heuristics for combinatorial optimization converge toward a single best algorithm or a small set of trade-off solutions. When the problem distribution shifts, these methods require expensive re-adaptation and their performance degrades. Here, we propose a quality-diversity evolutionary framework that maintains a structured archive of diverse heuristics, where each is specialized for a particular class of problem instances. A large language model serves as the variation operator within the evolutionary loop. Shapley additive explanations on abstract syntax tree features play a dual role in this framework: they guide the mutation prompts toward productive code modifications, and they extract the behavioral descriptors that organize the archive. An online selector then retrieves the best-matched heuristic for each new unseen instance. We evaluate the approach on vehicle routing, traveling salesman, and job shop scheduling problems under static, perturbed, stochastic, and dynamic conditions. The archive coverage provides inherent robustness to distributional shift, whereas single-heuristic baselines and neural solvers degrade considerably. The resulting heuristic library is interpretable, since each archive cell contains readable code at known feature coordinates.**

***Index Terms*—Quality-diversity optimization, automated heuristic design, evolutionary computation, combinatorial optimization, explainable artificial intelligence.**

I. Introduction

Combinatorial optimization (CO) problems—including capacitated vehicle routing (CVRP), traveling salesman (TSP), and job shop scheduling (JSSP)—arise across operations research and industrial engineering. Because these problems are NP-hard, exact solvers become intractable at scale, and evolutionary computation (EC) methods have long provided near-optimal solutions within practical time budgets [1]. EC methods, however, still rely on hand-crafted operators and tuned hyperparameters [2]; transfer across instances remains difficult [3], and classical metaheuristics rarely retain knowledge from one instance for the next [4].

Machine learning (ML) has therefore become a natural complement: data-driven methods can learn operators, configure hyperparameters online, and sometimes construct end-to-end solvers that generalize across instances.

Large language models (LLMs) have made automated heuristic design particularly active: FunSearch paired an LLM with evolutionary search to discover programs for mathematical combinatorics [5]; EoH transferred this idea to CO by iteratively refining heuristic code [6]; ReEvo added reflective mutation [7]; and LLaMEA evolved complete metaheuristic algorithms [8].

Further work has addressed specific limitations of these first-generation frameworks. DASH reconceptualized the evaluation of evolved solvers through a dynamics-aware lens and introduced a trajectory-based convergence metric that co-optimizes both the search mechanism and its runtime schedule [9]. MEoH formulated heuristic evolution as a multi-objective problem, where solution quality, runtime, and code complexity are balanced through a dominance-dissimilarity mechanism [10]. VRPAgent [11] nested LLM-generated operators inside a large neighborhood search shell to enforce constraint feasibility, while HeurAgenix [12] combined contrastive evolution with an adaptive online selector. G-LNS used the LLM to generate structural large neighborhood search operators that reshape the search landscape [13]. LLaMEA-SAGE incorporated explainable AI feedback—specifically, Shapley additive explanations (SHAP) [14] computed on abstract syntax tree (AST) features—to guide the LLM's mutation process toward productive code modifications [15].

Despite these advances, current LLM-driven heuristic design methods share a structural limitation: they converge toward one best heuristic or a small Pareto front tuned to the training distribution. When that distribution shifts, performance can degrade sharply. Neural CO solvers such as the Attention Model [16] and POMO [17] show the same fragility, losing more than 20% solution quality on realistic stochastic benchmarks after training on synthetic 100-node instances [18]. Re-adaptation strategies, such as DASH's profiled library retrieval [9], reduce the transition cost but do not remove distribution specificity.

Quality-diversity (QD) algorithms offer a principled alternative. Rather than converge to a single optimum, QD methods maintain a structured archive of diverse, high-performing solutions across behavioral niches [19], [20]. MAP-Elites [19] partitions a user-defined behavioral space into a grid and retains the elite solution in each cell, revealing how fitness varies across the space instead of collapsing it to one point. QD methods have proven effective in robotics, game design, and prompt engineering for language models, yet they have not been applied to the automated design of heuristic algorithms for combinatorial optimization.

We address this open problem by introducing a framework,

(Corresponding author: Shikun Chen.)

Shikun Chen and Yangguang Liu are with the College of Artificial Intelligence, Ningbo University of Finance & Economics, Ningbo 315175, China (e-mail: chenshikun@nbufe.edu.cn; liuyangguang@nbufe.edu.cn).

ORCID: 0000-0001-5958-7545 (Shikun Chen); 0000-0001-9468-9651 (Yangguang Liu).

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

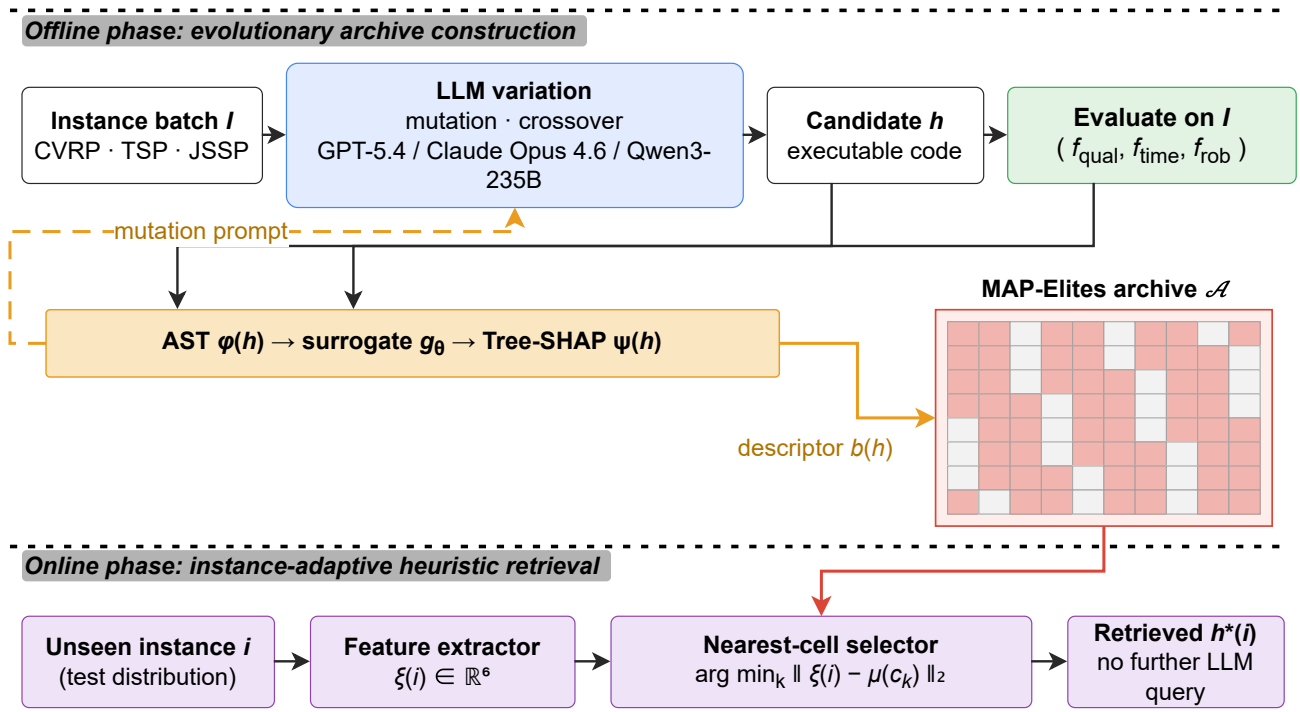


Fig. 1. Architecture of the proposed QD-LHD framework. An LLM generates candidate heuristics as executable code within a MAP-Elites evolutionary loop. Each candidate is evaluated on problem instances and placed in a structured archive indexed by behavioral descriptors that are extracted via SHAP analysis of abstract syntax tree features. SHAP values simultaneously guide the LLM's mutation prompts, which creates an explainable, self-organizing heuristic library. At deployment, an online selector retrieves the best-matched heuristic for each new instance based on its feature coordinates.

hereafter referred to as QD-LHD, that integrates MAP-Elites population management with LLM-driven heuristic generation and explainable AI feedback. As illustrated in Fig. 1, an LLM generates executable heuristic code, each candidate is evaluated on problem instances, and MAP-Elites places it according to behavioral descriptors extracted from the code's abstract syntax tree through SHAP analysis of a surrogate performance model [14]. SHAP values therefore define archive coordinates and also inform the natural-language prompts for later LLM mutations. At deployment, an online selector maps each unseen instance to an archive cell and retrieves the corresponding heuristic without additional LLM queries.

The contributions of this work are as follows:

- We reframe LLM-driven heuristic design as a quality-diversity problem: rather than searching for one good heuristic, the evolutionary loop fills an archive of specialized heuristics indexed by a behavioral space. The resulting framework, QD-LHD, generalizes both single-objective methods such as EoH [6] and multi-objective methods such as MEoH [10], and gives deployment-time retrieval a structured target.
- We introduce a dual-purpose use of SHAP analysis [14]: as mutation guidance for the LLM—an extension of the feedback mechanism in LLaMEA-SAGE [15] to combinatorial optimization—and as the behavioral descriptor extractor that organizes the archive.
- We establish a cell-decomposed robustness bound (Proposition 4.3) that connects archive coverage to worst-case performance under distributional shift: the archive replaces a single global Lipschitz constant with a weighted sum of cell-local constants, controlled on regions where each specialized heuristic stays feasible.
- We evaluate QD-LHD on vehicle routing, traveling salesman, and job shop scheduling problems under static, perturbed, stochastic, and dynamic conditions. Experiments demonstrate that QD-LHD outperforms single-heuristic baselines and neural solvers, in particular under distributional shift.

The remainder of this paper reviews related work, presents QD-LHD, states three analytical properties, reports experiments, and concludes.

II. Related Work

This work draws on LLM-driven heuristic design, quality-diversity evolutionary algorithms, and explainable AI for algorithm design.

A. LLM-Driven Heuristic Design

Treating programs as the unit of optimization has roots in genetic programming, where syntax trees are evolved to produce human-competitive algorithms. Hyper-heuristics extended this paradigm to evolve selection or construction heuristics over low-level metaheuristics [3], [21]. These approaches, however, are constrained by fixed grammars and limited code representations; recent surveys cover the broader landscape of LLM-driven [22] and meta-learning-assisted [23] algorithm design.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

Large language models remove this constraint by generating, modifying, and reasoning about code in natural language. FunSearch [5] paired an LLM with an evolutionary population of programs and discovered extremal-combinatorics constructions beyond prior human results. EoH [6] transferred this idea to CO, ReEvo [7] added reflective self-analysis before mutation, and LLaMEA [8] broadened the scope to complete metaheuristic algorithms.

DASH [9] introduced a dynamics-aware evaluation that co-optimizes the heuristic and its runtime schedule, reducing required LLM queries through trajectory-based convergence detection. MEoH [10] cast heuristic evolution as a multi-objective problem in which solution quality, runtime, and code complexity are balanced through a dominance-dissimilarity archive. VRPAgent [11] embedded LLM-generated destroy-and-repair operators inside a large neighborhood search shell to guarantee constraint feasibility on vehicle routing. HeurAgenix [12] introduced contrastive heuristic evolution paired with an adaptive online selector. G-LNS [13] used the LLM to produce structural neighborhood search operators that reshape the search landscape. LLaMEA-SAGE [15] incorporated SHAP-based feedback on abstract syntax tree features to steer the mutation process toward productive code modifications.

Reinforcement learning (RL) offers a parallel route to automated algorithm design, pursued in depth by the MetaEvo group[1] [23]: ConfigX [24] meta-learns a universal configuration policy over a modular space of evolutionary operators, DesignX [25] assembles complete optimizers from that space, and Q-Mamba [26] casts dynamic algorithm configuration as offline Q-function learning. The two families differ in three respects. RL designers search a predefined module library, whereas an LLM writes free-form code; they pay an offline meta-training cost and then configure unseen tasks in milliseconds, whereas LLM-based design needs no meta-training but spends API calls per run; and they return a neural policy, whereas the LLM returns readable source code—the property on which our SHAP-on-AST descriptors and auditable archive rest. The paradigms are complementary: the archive-plus-selector construction of QD-LHD could host RL-designed components, provided they are expressed as executable code.

Three recent lines sit particularly close to QD-LHD. MEoH [10] reformulates heuristic evolution as a multi-objective problem and returns a non-dominated front. In its native form, however, MEoH carries no mapping from instance characteristics to front members; the practitioner must choose a Pareto-front member before deployment. In our experiments we therefore report a strengthened variant, *MEoH with our deployment selector*, so the comparison does not penalize MEoH for lacking a retrieval rule. QD-LHD, by contrast, returns a behavioral-space archive whose cells carry affinity centroids by construction, and its third objective penalizes stochastic-realization variance rather than code complexity. HeurAgenix [12] combines contrastive heuristic evolution with an adaptive online selector over a generated pool. The pool in HeurAgenix grows without behavioral structure, and selection is based on instance features alone; there is no illumination mechanism that drives coverage of a heuristic-strategy space. EoH [6] and its selection-extended descendants share this feature: the underlying population is treated as a set of candidates rather than as an illuminated archive. The closest work conceptually is that of Liu et al. [27], which also generates a library of heuristics and selects one per instance. Two distinctions matter: (i) their library is partitioned by *instance* features, whereas our archive is partitioned by *code-structural* features derived from SHAP attributions on the AST; (ii) their generation process has no illumination guarantee, whereas MAP-Elites gives our construction a coverage bound (Proposition 4.1) that drives explicit archive occupancy. Table I summarizes the distinctions.

[1]https://metaevo.github.io

TABLE I
POSITION OF QD-LHD RELATIVE TO THE CLOSEST LLM-DRIVEN HEURISTIC-DESIGN METHODS. "POPULATION" DESCRIBES WHAT THE METHOD MAINTAINS; "INDEX" IS THE STRUCTURE USED TO RETRIEVE A HEURISTIC AT DEPLOYMENT; "XAI" INDICATES WHETHER EXPLAINABILITY FEEDBACK IS FED BACK INTO THE SEARCH LOOP.

Method	Population	Index	XAI
EoH [6]	single-obj. pop.	—	—
MEoH [10]	non-dominated set	—	—
HeurAgenix [12]	pool + selector	instance feat.	—
LLaMEA-SAGE [15]	single-obj. pop.	—	SHAP-AST
Liu et al. [27]	instance-spec. lib.	instance feat.	—
QD-LHD (ours)	MAP-Elites archive	SHAP-AST (code)	SHAP-AST

The three components of QD-LHD are coupled by construction: the archive needs a descriptor space to index cells, the SHAP-AST descriptors need the archive to accumulate evaluation data, and the online selector needs both to retrieve a heuristic per instance. Indexing the archive by code-structural behavior rather than by instance features or raw fitness is what gives the framework its deployment-time interpretability and its robustness decomposition in Section IV.

B. Quality-Diversity Algorithms

Quality-diversity (QD) algorithms use both fitness and behavioral diversity as selection criteria [20]. MAP-Elites [19] partitions a user-defined behavioral space into a grid and stores the highest-performing solution in each cell. CVT-MAP-Elites [28] replaces the fixed grid with centroidal Voronoi tessellations for high-dimensional spaces, while CMA-ME [29] combines MAP-Elites with covariance matrix adaptation.

Cully et al. [30] demonstrated that a MAP-Elites archive enables a damaged robot to adapt within minutes by retrieval from a pre-computed locomotion repertoire without retraining. CycleQD [31] used quality-diversity search to diversify the skills of LLM-based agents.

MAP-Elites naturally produces the library structure needed for an instance-adaptive heuristic portfolio, and its behavioral descriptor space provides the indexing mechanism that existing LLM-driven methods lack. The question of how to define such descriptors for heuristic code, however, has not been addressed in the QD literature; we resolve this through SHAP-based feature extraction from abstract syntax trees.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

C. Explainable AI for Algorithm Design

Explainable AI (XAI) techniques have also been applied to algorithm design and selection. Exploratory landscape analysis (ELA) [32] extracts numerical landscape features, and Kerschke and Trautmann [33] showed their predictive power for algorithm performance across continuous benchmarks.

In the context of evolutionary computation, XAI methods have been used to explain algorithm behavior rather than to guide it. SHAP [14] can attribute the performance of a surrogate model to individual features, and such attribution has been used to analyze why certain algorithm configurations outperform others. The feedback, however, has remained post-hoc: it informs the human analyst, not the algorithm itself.

LLaMEA-SAGE [15] closed this loop by feeding SHAP attributions on AST features back to the LLM as natural-language mutation guidance. It was demonstrated on continuous black-box optimization with a single-objective population.

Our framework extends this feedback in two directions. First, SHAP values guide mutation and define the behavioral descriptors that organize the MAP-Elites archive. Second, we apply this mechanism to combinatorial optimization, where discrete constraints pose challenges distinct from continuous optimization.

III. The QD-LHD Framework

This section presents the proposed framework in full. We first formulate the optimization objective and introduce notation. We then describe the archive structure, the SHAP-based behavioral descriptor mechanism, the LLM-driven variation operators, the multi-objective cell competition rule, and the online selector. Algorithm 2 summarizes the complete evolutionary loop.

A. Problem Formulation

Definition 3.1 (Heuristic Program): A heuristic program $h \in \mathcal{H}$ is an executable function that accepts a combinatorial optimization instance $i \in \mathcal{I}$ and returns a feasible solution with objective cost $\mathrm{cost}(h(i), i) \in \mathbb{R}_{\geq 0}$.

We evaluate each heuristic on a representative instance set $I \subset \mathcal{I}$ through three runtime-measured objectives. The *quality* objective measures the average optimality gap:

$$f_{\mathrm{qual}}(h, I) = \frac{1}{|I|} \sum_{i \in I} \frac{\mathrm{cost}(h(i), i) - \mathrm{cost}^*(i)}{\mathrm{cost}^*(i)}, \tag{1}$$

where $\mathrm{cost}^*(i)$ denotes the best known solution cost for instance i. The *efficiency* objective records the average wall-clock execution time:

$$f_{\mathrm{time}}(h, I) = \frac{1}{|I|} \sum_{i \in I} t(h, i), \tag{2}$$

where $t(h, i)$ is the runtime of h on instance i. The *robustness* objective quantifies performance variability under stochastic perturbations:

$$f_{\mathrm{rob}}(h, I) = \frac{1}{|I|} \sum_{i \in I} \mathrm{CV}\big[\mathrm{cost}(h(i^{(r)}), i^{(r)})\big]_{r=1}^{R}, \tag{3}$$

where $i^{(1)}, \ldots, i^{(R)}$ are R stochastic realizations of instance i and $\mathrm{CV}[\cdot]$ is the coefficient of variation. For deterministic instances, f_{rob} is set to zero. All three objectives are to be minimized, and all three are observable only by executing the code—unlike the static AST features of Table II, which can be read off the source alone. Two heuristics with nearly identical AST feature vectors can still differ markedly in wall-clock cost or realization variance, so f_{time} and f_{rob} sit at the selection layer while $\phi(h)$ remains a structural view of the source: the AST features index what the code looks like, the objectives index how it behaves when run. We write the combined vector as $\mathbf{f}(h, I) = (f_{\mathrm{qual}}, f_{\mathrm{time}}, f_{\mathrm{rob}})$.

Definition 3.2 (Archive): The behavioral descriptor space $\mathcal{B} = [0, 1]^d$ is partitioned into $K = B^d$ cells $\{c_1, \ldots, c_K\}$ by a uniform grid with B bins per dimension. The archive $\mathcal{A} : \{c_1, \ldots, c_K\} \to \mathcal{H} \cup \{\emptyset\}$ maps each cell to at most one elite heuristic.

Definition 3.3 (QD-Score): The quality-diversity score of an archive is

$$\mathrm{QD}(\mathcal{A}, I) = \sum_{k=1}^{K} \max\big(0,\ 1 - f_{\mathrm{qual}}(\mathcal{A}(c_k), I)\big) \cdot \mathbb{1}[\mathcal{A}(c_k) \neq \emptyset], \tag{4}$$

where the $\max$ operator clamps the per-cell quality score to $[0, 1]$. The QD-Score simultaneously rewards high solution quality and broad archive coverage. Empty cells contribute zero to the sum.

The objective of QD-LHD is to produce an archive $\mathcal{A}^*$ that maximizes $\mathrm{QD}(\mathcal{A}^*, I)$ while minimizing f_{time} and f_{rob} within each occupied cell.

B. SHAP-Derived Behavioral Descriptors

The behavioral descriptor function $b : \mathcal{H} \to \mathcal{B}$ determines each heuristic's position in the archive. In prior QD work, this function is specified manually from domain knowledge [19], [20]. We instead derive it automatically from the code structure of each heuristic through a two-stage process: AST feature extraction and SHAP attribution. The archive is organized around code structure because its purpose is to illuminate *the algorithm* space; instance characteristics enter the framework at the downstream retrieval step of Section III-E. The two layers therefore play distinct roles: the archive organizes heuristics by how they are written, and the selector maps each instance to the heuristic whose structural niche suits it best.

1) AST Feature Extraction: Each heuristic h is parsed into an AST, from which a feature vector $\phi(h) = (\phi_1(h), \ldots, \phi_p(h)) \in \mathbb{R}^p$ is extracted. Table II lists the $p = 11$ features used in our implementation. They capture code structure at multiple levels of granularity: token-level statistics, control-flow patterns, and algorithmic signatures such as recursion and sorting usage.

2) Surrogate Model and SHAP Attribution: A gradient-boosted tree model $g_\theta : \mathbb{R}^p \to \mathbb{R}$ is trained on the accumulated evaluation data $\mathcal{D} = \{(\phi(h_i), f_{\mathrm{qual}}(h_i, I))\}_{i=1}^{N}$, where N is the number of heuristics evaluated so far. We use XGBoost [34] as the surrogate for its balance of predictive accuracy and computational speed.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

TABLE II
AST FEATURES EXTRACTED FROM EACH HEURISTIC PROGRAM. THESE FEATURES SERVE AS INPUT TO THE SURROGATE PERFORMANCE MODEL.

Category	Feature	Description
Structure	`ast_depth`	Maximum AST depth
	`num_nodes`	Total AST node count
	`code_lines`	Lines of executable code
Control flow	`num_loops`	Loop construct count
	`loop_nesting`	Max loop nesting depth
	`num_branches`	Conditional branch count
Operations	`num_arith`	Arithmetic operator count
	`num_compare`	Comparison operator count
Algorithmic	`has_recursion`	Uses recursive calls (0/1)
	`has_sorting`	Uses sort operations (0/1)
	`num_ds_types`	Distinct data structure types

For each heuristic h, Tree-SHAP [14], [35] computes the Shapley value of each AST feature:

$$\psi_j(h) = \sum_{S \subseteq P \setminus \{j\}} \frac{|S|!\,(p-|S|-1)!}{p!} \big[g_\theta(\phi_{S\cup\{j\}}) - g_\theta(\phi_S) \big], \tag{5}$$

where $P = \{1, \ldots, p\}$ is the full feature set and ϕ_S denotes the feature vector with features outside S marginalized over the training distribution. The SHAP values satisfy the efficiency property $\sum_{j=1}^{p} \psi_j(h) = g_\theta(\phi(h)) - \mathbb{E}[g_\theta]$, so individual attributions sum exactly to the model's prediction for that heuristic. Tree-SHAP computes (5) in $O(TL^2)$ time for an ensemble of T trees with at most L leaves each, and this polynomial cost avoids the exponential complexity of naive Shapley enumeration.

3) Behavioral Descriptor Construction: The d AST features with the highest mean absolute SHAP value across the archive population are selected as the behavioral dimensions:

$$\{j_1, \ldots, j_d\} = \underset{j \in P}{\arg\,\text{top-d}} \; \frac{1}{|\mathcal{A}|} \sum_{h \in \mathcal{A}} |\psi_j(h)|. \tag{6}$$

The behavioral descriptor of a heuristic h is then

$$b(h) = \left(\frac{\psi_{j_1}(h) - \psi_{j_1}^{\min}}{\psi_{j_1}^{\max} - \psi_{j_1}^{\min}}, \; \ldots, \; \frac{\psi_{j_d}(h) - \psi_{j_d}^{\min}}{\psi_{j_d}^{\max} - \psi_{j_d}^{\min}} \right), \tag{7}$$

where $\psi_j^{\min}$ and $\psi_j^{\max}$ are the minimum and maximum SHAP values for feature j observed across the archive. This min-max normalization maps all descriptors to $[0,1]^d$.

The descriptor dimensions are recomputed every Δ generations to reflect the state of the evolving surrogate. In practice, the top-d features stabilize after the surrogate has observed a few hundred heuristics (see Section V). This mechanism differs from the standard MAP-Elites approach in which descriptors are fixed a priori. Here, the archive self-organizes as the surrogate improves: the AST features that best predict performance variation become the axes along which diversity is maintained. Two heuristics with distinct code structure therefore land in different cells regardless of instance similarity; the QD-LHD-HD ablation in Section V-E quantifies this effect.

C. LLM-Driven Variation Operators

The LLM serves as both the initialization mechanism and the variation operator within the evolutionary loop.

Assumption 3.1 (LLM Generation): The LLM $\mathcal{M}$ accepts a natural-language prompt that contains source code, performance metrics, and structural feedback, and returns a modified heuristic program. The generated code is syntactically valid with probability $p_{\text{valid}} > 0$, and the content of the prompt influences but does not deterministically control the distribution of generated programs.

1) Seed Generation: The archive is seeded with $K_{\text{init}} = 50$ heuristics, drawn in five persona groups of ten: greedy construction, local search with neighborhood exploration, large-neighborhood destroy-and-repair, constraint-first decomposition, and hybrid compositions of these. Each seed prompt concatenates three blocks—a persona directive (*"act as an algorithm designer who favors greedy construction"*), a target instance profile (*"design a construction heuristic for large-scale routing instances with tight capacity constraints"*), and the required function signature—so the LLM receives the stylistic prior, the problem context, and the interface in a single template. No performance feedback or SHAP information is supplied at this phase. Because the surrogate model has not yet been trained, the behavioral descriptor is read directly from the raw AST feature vector: the three features with the largest observed variance across the seed batch are min-max normalized to $[0,1]$ and serve as the descriptor axes until the first surrogate retraining at generation Δ.

2) SHAP-Guided Mutation: At each generation, a parent h_p is sampled uniformly at random from the occupied cells of the archive. The mutation operator constructs a prompt with four components: (i) the parent's source code, (ii) its performance vector $\mathbf{f}(h_p, I)$, (iii) SHAP-derived structural feedback, and (iv) an optional target behavioral niche—the descriptor coordinates of a sparsely-populated neighboring cell, injected only during coverage-recovery phases (when archive growth has plateaued) to bias the LLM toward that niche, and omitted otherwise.

The SHAP feedback is constructed from the parent's attribution vector $\boldsymbol{\psi}(h_p)$. Features whose SHAP value exceeds the 75th percentile threshold τ^+ are reported as positive contributors; those below the 25th percentile threshold τ^- are identified as detrimental. This information is verbalized in natural language—for instance:

"Loop nesting depth (SHAP $= +0.12$) contributes positively to quality. Comparison operator count (SHAP $= -0.08$) degrades performance. Simplify the conditional logic while preserving the nested loop structure."

Algorithm 1 details the procedure, and Fig. 2 presents a concrete example for capacitated vehicle routing.

3) Crossover: Two parents h_a and h_b are selected from different archive cells. The LLM receives both source codes together with a SHAP comparison that highlights which structural features of each parent are most beneficial for a specified target niche. The prompt instructs $\mathcal{M}$ to merge complementary strengths from both parents into a single offspring.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

(a) SHAP-Guided Mutation Prompt

[Parent Heuristic] — *Greedy nearest-neighbor for CVRP*

```
def heuristic(inst):
    routes = [[0]]
    unvis = set(range(1, inst.n))
    cap = inst.capacity
    while unvis:
        cur = routes[-1][-1]
        best, bd = None, float('inf')
        for v in unvis:
            d = inst.dist(cur, v)
            if d < bd and inst.demand[v] <= cap:
                best, bd = v, d
            elif d < bd and len(unvis) > inst.n//4:
                continue
            elif d < bd:
                best, bd = v, d
        if best is None:
            routes.append([0])
            cap = inst.capacity
        else:
            routes[-1].append(best)
            unvis.discard(best)
            cap -= inst.demand[best]
    return routes
```

[Performance] Optimality gap: 12.3%, Time: 0.05 s, CV: 0.08

[SHAP Structural Feedback]

+ `loop_nesting` (SHAP $= +0.15$): Nested loops contribute positively to solution quality.

− `num_branches` (SHAP $= -0.11$): Excessive conditional branching degrades performance.

+ `has_sorting` (SHAP $= +0.09$): Sorting-based construction correlates with better solutions.

[Instruction] Modify the heuristic to improve quality. Reduce conditional branches while preserving the nested loop structure. Consider adding a sorting-based construction step.

(b) LLM Response

[Generated Offspring Heuristic]

```
def heuristic(inst):
    # Sort by demand-to-distance ratio
    nodes = sorted(range(1, inst.n),
        key=lambda v:
            -inst.demand[v] / inst.dist(0, v))
    routes, cap = [[0]], inst.capacity
    for v in nodes:
        if inst.demand[v] <= cap:
            routes[-1].append(v)
            cap -= inst.demand[v]
        else:
            routes.append([0, v])
            cap = inst.capacity - inst.demand[v]
    # 2-opt local search per route
    for route in routes:
        improved = True
        while improved:
            improved = False
            for i in range(1, len(route) - 1):
                for j in range(i+1, len(route)):
                    if swap_gain(route,i,j) > 0:
                        route[i:j+1] = reversed(
                            route[i:j+1])
                        improved = True
    return routes
```

[Structural Changes]

- Replaced nearest-neighbor scan (3 conditional branches) with sorted insertion (1 branch): `num_branches` $6 \rightarrow 2$.
- Added sorting-based construction: `has_sorting` $0 \rightarrow 1$.
- Added 2-opt with deeper nesting: `loop_nesting` $2 \rightarrow 3$.
- Offspring SHAP profile $(0.71, 0.31, 0.55)$ differs from parent $(0.62, 0.83, 0.10)$, and thus the offspring is placed in a different archive cell.

Fig. 2. Concrete example of SHAP-guided LLM mutation for capacitated vehicle routing. (a) The mutation prompt includes the parent heuristic's source code, its evaluation performance, SHAP attributions that identify which AST features help or hurt quality, and a natural-language instruction derived from this feedback. (b) The LLM generates a modified heuristic that addresses the SHAP guidance: conditional branches are reduced, a sorting step is introduced, and 2-opt local search with deeper loop nesting is added. The offspring's SHAP profile places it in a different archive cell than the parent.

Algorithm 1 SHAP-Guided LLM Mutation

Require: Parent h_p, surrogate g_θ, feature map ϕ, LLM $\mathcal{M}$, thresholds τ^+, τ^-
Ensure: Offspring h'

$\mathbf{x} \leftarrow \phi(h_p)$ ▷ Extract AST features
$\boldsymbol{\psi} \leftarrow \textsc{TreeSHAP}(g_\theta, \mathbf{x})$
$F^+ \leftarrow \{j : \psi_j > \tau^+\}$; $F^- \leftarrow \{j : \psi_j < \tau^-\}$
feedback $\leftarrow \textsc{Verbalize}(F^+, F^-, \boldsymbol{\psi})$
prompt $\leftarrow \textsc{Compose}(h_p.\text{code}, \mathbf{f}(h_p), \textit{feedback})$
$h' \leftarrow \mathcal{M}.\text{generate}(\textit{prompt})$
if h' passes syntax and execution checks **then**
  return h'
else
  return $\emptyset$ ▷ Discard invalid offspring
end if

D. Multi-Objective Cell Competition

When an offspring h' is produced, its behavioral descriptor $b(h')$ determines a target cell c_k. If the cell is empty, h' is inserted directly. If it is occupied by an incumbent h_k^*, the replacement follows a two-stage rule.

Definition 3.4 (Pareto Dominance): Heuristic h dominates h', written $h \succ h'$, if and only if $f_m(h, I) \leq f_m(h', I)$ for all $m \in \{1, 2, 3\}$ with at least one strict inequality.

Stage 1. If $h' \succ h_k^*$, the offspring replaces the incumbent.

Stage 2. If neither heuristic dominates the other, the one with lower weighted scalarization is retained:

$$s(h) = w_1 f_{\text{qual}}(h, I) + w_2 f_{\text{time}}(h, I) + w_3 f_{\text{rob}}(h, I), \quad (8)$$

where the weights $w_1, w_2, w_3 \geq 0$ satisfy $w_1 + w_2 + w_3 = 1$. We set $w_1 = 0.6$, $w_2 = 0.2$, $w_3 = 0.2$ by default to prioritize solution quality. This two-stage rule preserves Pareto-dominant solutions while permitting turnover in non-dominated cases [36].

E. Online Instance-Adaptive Selection

At deployment, the archive serves as a heuristic library from which the best-matched algorithm is retrieved for each unseen instance, following the per-instance algorithm selection paradigm [37], [38]. This is the instance-feature layer of the framework: while the archive of Section III-B indexes heuristics by their code structure, the selector below indexes *instances* by their problem characteristics and matches each one to a cell. No further LLM queries are needed at deployment, which keeps the per-instance cost at $O(K)$ nearest-

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

neighbor lookups and leaves the heuristic library fully auditable; continual refinement of the archive is treated as a future extension in Section VI.

An instance feature extractor $\xi : \mathcal{I} \to \mathbb{R}^{d'}$ maps each problem instance to a vector of $d' = 6$ characteristics: (1) problem size n (customer nodes for CVRP/TSP, jobs $\times$ machines for JSSP); (2) constraint density ρ, instantiated as $\sum_i d_i/(K_{\text{veh}}Q)$ for CVRP/VRPTW, $\sum_{i,j} p_{ij}/(m\,\text{LB})$ for JSSP, and $\rho \equiv 1$ for TSP, where K_{veh} is the vehicle count and Q is vehicle capacity; (3) demand or weight variability $\text{CV}(d_i)$ for CVRP, $\text{CV}(p_{ij})$ for JSSP, and 0 for TSP; (4) cost matrix mean $\bar{c}$ over non-zero entries; (5) cost matrix coefficient of variation $\text{CV}(c_{ij})$; and (6) graph clustering coefficient on the k_{nn}-nearest-neighbor graph of customer or job nodes. All features are min-max normalized to $[0, 1]$ using the training set statistics, with test-time features clipped to $[0, 1]$ on the training support so that out-of-range values at deployment do not propagate into the selector's cell assignment. During the evolution phase, we record the *instance affinity* of each occupied cell. For cell c_k with elite h_k^*, the affinity centroid is

$$\mu(c_k) = \frac{1}{|I_k|} \sum_{i \in I_k} \xi(i), \tag{9}$$

where $I_k \subseteq I$ is the subset of training instances on which h_k^* achieves the lowest cost among all archive elites.

Given a new instance i, the selector retrieves:

$$h^*(i) = \mathcal{A}\left(\underset{c_k :\, \mathcal{A}(c_k) \neq \emptyset}{\arg\min} \|\xi(i) - \mu(c_k)\|_2 \right). \tag{10}$$

When the nearest occupied cell lies beyond a distance threshold δ—that is, $\min_k \|\xi(i) - \mu(c_k)\| > \delta$—the selector falls back to the heuristic from whichever of the three nearest occupied cells has the lowest scalarized objective $s(h)$ on its own affinity instances. This fallback provides graceful degradation for instances that lie outside the archive's covered region.

The selection procedure is deterministic and requires $O(K)$ nearest-neighbor lookups, a cost that is negligible compared to the execution of the retrieved heuristic itself.

Algorithm 2 QD-LHD: Quality-Diversity LLM-Driven Heuristic Design

Require: Instance set I, LLM $\mathcal{M}$, descriptor dims d, bins B, generations G, mutation ratio ρ, update interval Δ
Ensure: Archive $\mathcal{A}$

Initialize $\mathcal{A}$ with $K = B^d$ empty cells
▷ *Phase 1: Seed generation*
for $k = 1, \ldots, K_{\text{init}}$ **do**
  $h_k \leftarrow \mathcal{M}$.generate(seed prompt with persona k)
  Evaluate $\mathbf{f}(h_k, I)$; extract $\phi(h_k)$
  Insert h_k into $\mathcal{A}$ via raw AST-based descriptor
end for
Train surrogate g_θ on $\{(\phi(h_i), f_{\text{qual}}(h_i, I))\}$
Select descriptor dims $\{j_1, \ldots, j_d\}$ via (6)
▷ *Phase 2: Evolutionary loop*
for $t = 1, \ldots, G$ **do**
  Sample parent h_p uniformly from occupied cells
  if $\text{rand}() < \rho$ **then** ▷ Mutation
    $h' \leftarrow \text{SHAPMUTATION}(h_p, g_\theta, \phi, \mathcal{M})$
  else ▷ Crossover
    Sample second parent h_q from a different cell
    $h' \leftarrow \text{SHAPCROSSOVER}(h_p, h_q, g_\theta, \phi, \mathcal{M})$
  end if
  if $h' \neq \emptyset$ **then**
    Evaluate $\mathbf{f}(h', I)$; extract $\phi(h')$
    $\boldsymbol{\psi} \leftarrow \text{TREESHAP}(g_\theta, \phi(h'))$
    $b(h') \leftarrow$ normalize $(\psi_{j_1}, \ldots, \psi_{j_d})$ via (7)
    $c \leftarrow \text{CELLOF}(b(h'), \mathcal{A})$
    if $\mathcal{A}(c) = \emptyset$ **or** $h' \succ \mathcal{A}(c)$ **or** $s(h') < s(\mathcal{A}(c))$ **then**
      $\mathcal{A}(c) \leftarrow h'$
    end if
  end if
  if $t \bmod \Delta = 0$ **then**
    Retrain g_θ; recompute $\{j_1, \ldots, j_d\}$ via (6)
  end if
end for
Compute instance affinities $\mu(c_k)$ for all occupied cells via (9)
return $\mathcal{A}$

IV. Analytical Properties

We establish three analytical properties of the framework: a coverage bound on the archive, a drift bound on the behavioral descriptors, and a robustness bound under distributional shift that decomposes over archive cells.

A. Archive Coverage

Proposition 4.1 (Coverage rate): Suppose that for every cell c_k and every parent h_p, a single application of the LLM variation operator produces an offspring whose descriptor falls in c_k with probability at least $p_{\min} > 0$. Let $C(G)$ denote the fraction of occupied cells after G generations. Then

$$\mathbb{E}[C(G)] \;\geq\; 1 - (1 - p_{\min})^G. \tag{11}$$

Proof. Fix a cell c_k. The probability it remains empty after G trials is at most $(1 - p_{\min})^G$. Hence its occupation probability is at least $1 - (1 - p_{\min})^G$. Averaging this lower bound over the K cells gives (11). □

The bound is conservative: it treats each cell's reachability probability as uniformly lower-bounded and ignores the directional bias introduced by SHAP-guided mutation. Its value is qualitative—the exponential dependence on G and the practical dependence of $p_{\min}$ on archive granularity motivate the archive-sizing study in Section V-E—rather than quantitative. Empirical coverage curves appear in Fig. 6.

B. Descriptor Stability

The archive's behavioral descriptors depend on SHAP values that are recomputed whenever the surrogate g_θ is retrained. A natural question is how much the descriptors of a fixed

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

heuristic drift between two successive retrainings. The following proposition gives a stability condition in terms of the fitted tree ensemble itself rather than only its scalar prediction: two models can agree on a prediction while attributing that prediction to different features, so attribution stability must account for leaf values and split structure.

Proposition 4.2 (Descriptor drift inherits from surrogate drift): Let $g_\theta^{(N_1)}$ and $g_\theta^{(N_2)}$ be two successive surrogates trained on N_1 and N_2 samples. For any heuristic h, suppose the two fitted tree ensembles have the same split topology in the leaves reached by $\phi(h)$, and let Δ_{leaf} be the maximum absolute change in the corresponding leaf values and path-cover weights used by Tree-SHAP. Then the Tree-SHAP attributions satisfy

$$\left\|\boldsymbol{\psi}^{(N_2)}(h) - \boldsymbol{\psi}^{(N_1)}(h)\right\|_2 \le \kappa\, \Delta_{\mathrm{leaf}}, \tag{12}$$

for a constant κ determined by the local tree topology and the number of AST features.

Proof. Tree-SHAP returns the exact Shapley values of the fitted ensemble [35]. For a fixed local split topology, each attribution is a finite weighted sum of leaf-value differences, with weights determined by the path-cover statistics. Perturbing those leaf values and weights by at most Δ_{leaf} perturbs the resulting attribution vector by at most a topology-dependent constant times Δ_{leaf}. □

The statement is local: if retraining changes the split topology around $\phi(h)$, attribution rankings can change discontinuously. Fig. 6(c) therefore reports the empirical counterpart of this stability condition: the top-d SHAP ranking stabilizes after roughly 250 evaluated heuristics.

C. Archive-Based Robustness

The final property concerns performance under distributional shift between training and deployment. Heuristic performance in combinatorial optimization is not globally smooth in instance features: a small change in instance geometry or constraint density can render a previously feasible route infeasible and produce a discontinuous jump in the optimality gap. A global Lipschitz condition on performance is therefore not an appropriate primitive in this setting. We instead impose a local Lipschitz condition, restricted to the feasibility support of each archive heuristic, and let the archive's selector handle transitions across feasibility boundaries.

Definition 4.1 (Feasibility-restricted support): For each occupied cell c_k with elite h_k^*, let $\mathcal{X}_k \subseteq \mathcal{X}$ denote the set of instance-feature vectors on which h_k^* returns a feasible solution.

A local Lipschitz condition on each heuristic's feasibility-restricted support asks h_k^* to behave smoothly only on the region where it remains feasible; discontinuities at the feasibility boundary are not folded into the Lipschitz constant. For an instance close to the boundary of $\mathcal{X}_k$, the selector in (10) routes it to the cell whose centroid is closest to $\xi(i)$, and the affinity construction in (9) biases that cell toward heuristics that are feasible near the instance. Instances outside the feasible region of the selected heuristic are handled by the fallback rule of Section III and enter the bound below through a selector-coverage residual.

Proposition 4.3 (Cell-decomposed robustness): Let $\mathcal{A}$ be an archive whose selector covers the test distribution up to residual mass $\varepsilon_{\mathrm{sel}}$, where $\varepsilon_{\mathrm{sel}} = \mathcal{Q}(\{i : \xi(i) \notin \mathcal{X}_{s(i)}\})$ and $s(i)$ denotes the cell selected for instance i. Suppose that for each occupied cell c_k there exists $L_k > 0$ such that $|f_{\mathrm{qual}}(h_k^*, \{i\}) - f_{\mathrm{qual}}(h_k^*, \{i'\})| \le L_k \left\|\xi(i) - \xi(i')\right\|_2$ for every $i, i' \in \mathcal{X}_k$. For a test distribution $\mathcal{Q}$, let $\pi_k = \mathcal{Q}(\{i : h^*(i) = h_k^*,\, \xi(i) \in \mathcal{X}_k\})$ denote the covered mass routed to cell c_k, and let $\mathcal{Q}_k$ and $\mathcal{P}_k$ be the conditional test and training distributions on $\mathcal{X}_k$. Then

$$\mathbb{E}_{\mathcal{Q}}\left[f_{\mathrm{qual}}(h^*)\right] \le \sum_k \pi_k \Big[\mathbb{E}_{\mathcal{P}_k}\left[f_{\mathrm{qual}}(h_k^*)\right] + L_k\, W_1(\mathcal{P}_k, \mathcal{Q}_k)\Big] + \varepsilon_{\mathrm{sel}}\, M, \tag{13}$$

where the sum runs over occupied cells and M is the worst-case optimality gap over $\mathcal{H} \times \mathcal{X}$.

Proof. Split $\mathcal{Q}$ into the selector-covered part, where the selected heuristic is feasible on its assigned support, and an uncovered part of mass $\varepsilon_{\mathrm{sel}}$ that contributes at most $\varepsilon_{\mathrm{sel}} M$. On each $\mathcal{X}_k$, the cell-local Lipschitz condition combined with Kantorovich–Rubinstein duality for W_1 [39] gives $\mathbb{E}_{\mathcal{Q}_k}[f_{\mathrm{qual}}] \le \mathbb{E}_{\mathcal{P}_k}[f_{\mathrm{qual}}] + L_k W_1(\mathcal{P}_k, \mathcal{Q}_k)$. Summing with weights π_k and adding the uncovered residual yields (13). □

Equation (13) decomposes the robustness bound over cells. A single-heuristic baseline carries only one pair $(L, W_1(\mathcal{P}, \mathcal{Q}))$, where L is a global Lipschitz constant that must absorb transitions across feasibility boundaries. The cell decomposition replaces this global constant by the π_k-weighted average of cell-local constants L_k, each controlled on a region where the associated specialized heuristic remains feasible. This does not make the local constants directly computable, nor does archive occupancy alone guarantee instance-space coverage; it states the condition under which a populated archive can reduce the effect of distributional shift. The three terms on the right-hand side map directly to design choices: $\varepsilon_{\mathrm{sel}}$ shrinks when the selector's affinity regions cover more of the deployment distribution, the L_k's shrink as cells become behaviorally narrower, and $W_1(\mathcal{P}_k, \mathcal{Q}_k)$ measures the residual train–test shift within each selected cell.

V. Experiments

A. Experimental Setup

1) Benchmark Problems: We evaluate QD-LHD on three CO problem families—CVRP with time windows (VRPTW), TSP, and JSSP—under static, perturbed, stochastic, and dynamic conditions. Table III summarizes the instance sets.

The non-nominal benchmarks span three regimes, each with its own evaluation protocol. *Perturbed* instances (our TSPLib variant [44]) are constructed offline by adding Gaussian noise to node coordinates; each realization is a fixed instance, and robustness is measured across realizations. *Stochastic* instances (SVRPBench/VRPTW) carry random parameters—travel times and service durations—that are drawn freshly at each evaluation. *Dynamic* instances (dynamic JSSP) reveal

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

TABLE III
BENCHMARK PROBLEM INSTANCES USED IN THE EXPERIMENTS.

Problem	Instance Set	Count
CVRP / VRPTW	CVRPLib Set X [40]	100
	Solomon VRPTW [41]	56
	SVRPBench (stochastic) [18]	500
TSP	TSPLib [42]	111
	Perturbed TSP (node perturbation)	100
JSSP	Taillard [43]	80
	Dynamic JSSP (random arrivals)	60

problem data sequentially during execution. The three regimes probe different aspects of distributional shift and are reported separately in the following subsections.

For the non-nominal variants, each instance is evaluated on $R = 10$ independent realizations. Perturbed TSP instances are constructed from TSPLib by perturbing node coordinates with Gaussian noise ($\sigma = 0.05\times$ bounding box diagonal); we also report results at $\sigma = 0.10$ as a stronger-noise regime (Table VII). Dynamic JSSP instances introduce random job arrivals sampled from a Poisson process at rate $\lambda = 0.2$ jobs per time unit, consistent with online scheduling formulations in the literature [43]. For classical baselines on non-nominal variants, LKH-3 [45] is re-run independently on each realization and the results are aggregated; CP-SAT [46] re-solves the constraint model after each job arrival within the same time budget.

2) Baseline Methods: We compare against four tiers of baselines. *Tier 1 (classical metaheuristics)*: LKH-3 for TSP, HGS-CVRP [47] for routing, and Google OR-Tools CP-SAT for scheduling. *Tier 2 (neural solvers)*: the Attention Model [16], POMO [17], and DIFUSCO [48], all trained on 100-node instances and evaluated without fine-tuning on the test distributions. DIFUSCO in its original form targets TSP and MIS; for CVRP we apply the natural graph-based adaptation used by the authors' public code [48]—each instance is solved as edge selection on the demand-weighted graph, and the decoded edges are post-processed into capacity-feasible routes by a standard savings sweep. For JSSP, we use an analogous disjunctive-graph adaptation: DIFUSCO predicts pairwise operation-order edges, and a priority-list decoder repairs cycles and produces machine-feasible schedules. We deliberately evaluate neural solvers in a zero-shot transfer setting to test distributional robustness; fine-tuned results from the original papers would narrow the gap on static benchmarks but do not reflect deployment scenarios where the test distribution is unknown a priori. *Tier 3 (LLM-driven heuristic design)*: EoH [6], ReEvo [7], DASH [9], MEoH [10], HeurAgenix [12], and G-LNS [13]. All Tier 3 baselines use the same LLM backbone, evaluation budget, and training instance batch I (which includes both static and stochastic instances) as QD-LHD for a fair comparison; the MEoH row should be read as *MEoH with our deployment selector*: because MEoH returns a non-dominated set over (quality, runtime, code complexity) rather than a single heuristic, we select its deployment-time heuristic by the same per-instance rule of Eq. 10 used for QD-LHD, retrieving the front member whose performance centroid is closest in feature space to the test instance. *Tier 4 (automated algorithm selection)*: AAS-NSS, our implementation of the Neural Solver Selection framework of Gao et al. [49] trained over the six Tier 3 LLM-generated heuristic baselines. This pool matches QD-LHD's deployment interface: every candidate is executable heuristic code produced under the same LLM budget. Classical solvers and neural solvers are reported separately rather than inserted into this selector because they use different solver interfaces, training regimes, and runtime profiles; mixing them would turn the control into a heterogeneous solver-selection benchmark rather than a heuristic-library selection baseline. The instance encoder is a graph attention network on the instance graph; the selector is a classifier that maps the resulting instance embedding to one of the six Tier 3 heuristics. Training uses the same instance batch I as all other baselines, with cross-entropy against the best-performing Tier 3 pool member per instance as the label.

3) Evaluation Metrics: Six metrics are reported. *Optimality gap* f_{qual} (1) measures solution quality relative to the best known cost. *Runtime* f_{time} (2) records wall-clock execution time per instance. *Robustness* f_{rob} (3) captures performance variability across stochastic realizations. For QD-LHD and its ablation variants, we additionally report *archive coverage* $C(G)$ (fraction of occupied cells, as defined in Proposition 4.1), *QD-score* (4) (which jointly captures coverage and per-cell quality), and *code complexity* measured by the mean cyclomatic complexity and lines of code of the archive elites. To assess generalization, we report the *shift degradation ratio*: the relative increase in optimality gap when the test distribution differs from the training distribution.

4) Hyperparameters: The QD-LHD archive uses $d = 3$ behavioral dimensions with $B = 10$ bins each, which yields $K = 1{,}000$ cells. The evolutionary loop runs for $G = 500$ generations with mutation ratio $\rho = 0.7$. The archive is seeded with $K_{\text{init}} = 50$ heuristics. The surrogate is retrained every $\Delta = 50$ generations. SHAP feedback thresholds are set to the 75th and 25th percentiles of the attribution distribution. Scalarization weights are $w_1 = 0.6$, $w_2 = 0.2$, $w_3 = 0.2$. Each configuration is run 30 times with different random seeds; every run executes the full offline pipeline of Algorithm 2 end-to-end—seed generation, evolution through $G = 500$ generations, and descriptor recomputation—so the means and standard deviations reported throughout Section V capture the variance of the entire archive-construction process rather than only the deployment-time replication of a single archive.

5) Implementation: The LLM backbone is GPT-5.4 (temperature 0.8, max 2,048 output tokens). We also report results with Claude Opus 4.6 and Qwen3-235B-A22B as alternative backbones in the ablation study. All LLM costs are reported per independent archive-construction run unless explicitly stated otherwise. The surrogate model is XGBoost with 200 trees and maximum depth 6. All heuristic evaluations run on a 64-core AMD EPYC 7763 server with 256 GB RAM. Neural baselines use two NVIDIA RTX 4090 GPUs. The framework is implemented in Python 3.11; generated heuristics execute in sandboxed subprocesses with a 60-second timeout per instance, no network access, and a read-only filesystem view.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

TABLE IV
RESULTS ON VEHICLE ROUTING BENCHMARKS. GAP (%) IS THE MEAN OPTIMALITY GAP; TIME (S) IS THE MEAN TIME PER INSTANCE; CV IS THE COEFFICIENT OF VARIATION ACROSS STOCHASTIC REALIZATIONS. BEST VALUES ARE IN **BOLD**; UNDERLINING MARKS QD-LHD WHEN IT IS SECOND ONLY TO THE CLASSICAL SOLVER ON STATIC QUALITY. †: QD-LHD WINS ($p < 0.05$).

		CVRPLib		Solomon		SVRPBench		
	Method	Gap	Time	Gap	Time	Gap	Time	CV
Cls.	HGS-CVRP	**0.23**	38.5	**0.35**	51.2	$4.17^{\dagger}$	41.6	$0.19^{\dagger}$
Neural	AM	$6.84^{\dagger}$	**0.9**	$8.27^{\dagger}$	**1.1**	$15.63^{\dagger}$	**1.0**	$0.37^{\dagger}$
Neural	POMO	$4.31^{\dagger}$	1.3	$5.43^{\dagger}$	1.6	$12.28^{\dagger}$	1.5	$0.32^{\dagger}$
Neural	DIFUSCO	$5.52^{\dagger}$	3.7	$6.78^{\dagger}$	4.3	$13.74^{\dagger}$	4.0	$0.34^{\dagger}$
LLM-Driven	EoH	$2.91^{\dagger}$	1.5	$3.65^{\dagger}$	1.9	$7.53^{\dagger}$	1.7	$0.20^{\dagger}$
LLM-Driven	ReEvo	$2.63^{\dagger}$	1.7	$3.32^{\dagger}$	2.1	$6.92^{\dagger}$	1.9	$0.18^{\dagger}$
LLM-Driven	DASH	$1.85^{\dagger}$	2.0	$2.18^{\dagger}$	2.6	$5.14^{\dagger}$	2.3	$0.14^{\dagger}$
LLM-Driven	MEoH	$2.47^{\dagger}$	1.8	$2.95^{\dagger}$	2.3	$5.87^{\dagger}$	2.1	$0.16^{\dagger}$
LLM-Driven	HeurAgenix	$2.12^{\dagger}$	1.3	$2.53^{\dagger}$	1.7	$5.46^{\dagger}$	1.5	$0.15^{\dagger}$
LLM-Driven	G-LNS	$2.28^{\dagger}$	2.2	$2.84^{\dagger}$	2.8	$6.15^{\dagger}$	2.5	$0.17^{\dagger}$
AAS	AAS-NSS [49]	$1.95^{\dagger}$	1.7	$2.28^{\dagger}$	2.2	$4.78^{\dagger}$	1.9	$0.13^{\dagger}$
Ours	QD-LHD	1.74	1.6	2.07	2.0	**2.93**	1.8	**0.08**

6) Statistical Protocol: We report the Wilcoxon signed-rank test for pairwise comparisons († in the result tables). Effect sizes are measured by the Vargha–Delaney $\hat{A}_{12}$ statistic [50]. All tests use a significance level of $\alpha = 0.05$.

B. Results on Vehicle Routing

Table IV presents the results on the CVRP/VRPTW benchmarks. We organize the discussion around three questions: static performance, stochastic robustness, and the trade-off between solution quality and computational cost.

1) Static Performance: On the deterministic CVRPLib and Solomon benchmarks, HGS-CVRP achieves the lowest optimality gap, as expected from a mature metaheuristic with decades of problem-specific engineering. Among the learning-based methods, QD-LHD ranks first and DASH is closest. The neural solvers perform notably worse, especially on Solomon, where time window constraints pose additional difficulty for models trained without them.

The advantage over other LLM-driven methods on static instances is modest yet consistent ($p < 0.05$ against all Tier 3 baselines). For a fixed distribution, the archive's diversity offers less benefit than under shift, since the selector retrieves the best cell-matched heuristic without needing to span the full archive breadth.

2) Stochastic Robustness: The stochastic SVRPBench results reveal sharper separations. Fig. 3 visualizes the gap increase from CVRPLib (static) to SVRPBench (stochastic) for each method. Neural solvers degrade most severely, confirming the distributional fragility reported in prior work [18]. The LLM-driven baselines are more resilient, yet all still trail QD-LHD under stochastic travel times.

QD-LHD has the lowest gap and coefficient of variation on SVRPBench. The archive's stochastic-specialized cells contain heuristics that explicitly account for travel time variability, and the online selector routes stochastic instances to these cells.

TABLE V
ARCHIVE-LEVEL METRICS FOR QD-LHD ON THE CVRP/VRPTW BENCHMARKS. VALUES ARE AVERAGED OVER 30 RUNS (MEAN ± STD).

Metric	**Value**
Archive coverage (%)	73.2 ± 3.1
QD-score	614.8 ± 18.6
Mean cyclomatic complexity	8.4 ± 1.2
Mean lines of code	31.7 ± 4.3
Shift degradation ratio	0.68 ± 0.05

In contrast, single-heuristic methods must rely on whatever robustness the single evolved heuristic happens to possess.

HGS-CVRP also degrades sharply on stochastic instances, evidence that even the best classical solvers do not transfer gracefully to this setting.

3) Quality–Cost Trade-off: Runtime varies considerably across tiers. HGS-CVRP is much slower due to its iterative local search. Neural solvers are fastest but pay for this speed with quality. QD-LHD's per-instance runtime falls in the same range as other LLM-driven methods; note that this is the *deployment* cost of executing the retrieved heuristic, not the offline archive construction cost. A single GPT-5.4 archive-construction run consumes approximately 500 LLM API calls and costs $85; the 30-run statistical protocol therefore costs $2,550 for one benchmark setting. The archive requires no further LLM queries once built, so the cost is amortized over every future deployment; Qwen3 lowers the per-run figure to $23 (3.7× cheaper, Table XI).

4) Archive Quality and Interpretability: Table V reports the QD-specific metrics for the CVRP/VRPTW experiments. The archive occupies a large majority of cells and maintains high-quality elites across the behavioral space. The average complexity and code length remain within the range that a human practitioner can read and understand.

The shift degradation ratio confirms the same mechanism observed in Fig. 3: stochastic instances are routed to cells specialized for their feature profile rather than served by a single algorithm.

C. Results on the Traveling Salesman Problem

Table VI summarizes the results on the TSP benchmarks. The traveling salesman problem serves as a stress test for QD-LHD because the classical baseline, LKH-3, is exceptionally strong on deterministic instances. We examine whether QD-LHD can remain competitive on static benchmarks while retaining its robustness advantage on perturbed variants.

1) Static Performance: LKH-3 dominates the static TSPLib benchmark, reflecting decades of dedicated engineering. Among learning-based methods, QD-LHD achieves the lowest gap and DASH is closest. The margin over other LLM-driven baselines is narrower than on CVRP, which reflects the simpler constraint structure of the TSP relative to capacitated routing.

2) Perturbed Performance and Distributional Spread: The perturbed-TSP results expose much larger performance gaps. Fig. 4 presents box plots across the 100 perturbed instances for each method. Neural solvers exhibit the highest medians and widest interquartile ranges, reflecting their vulnerability to

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

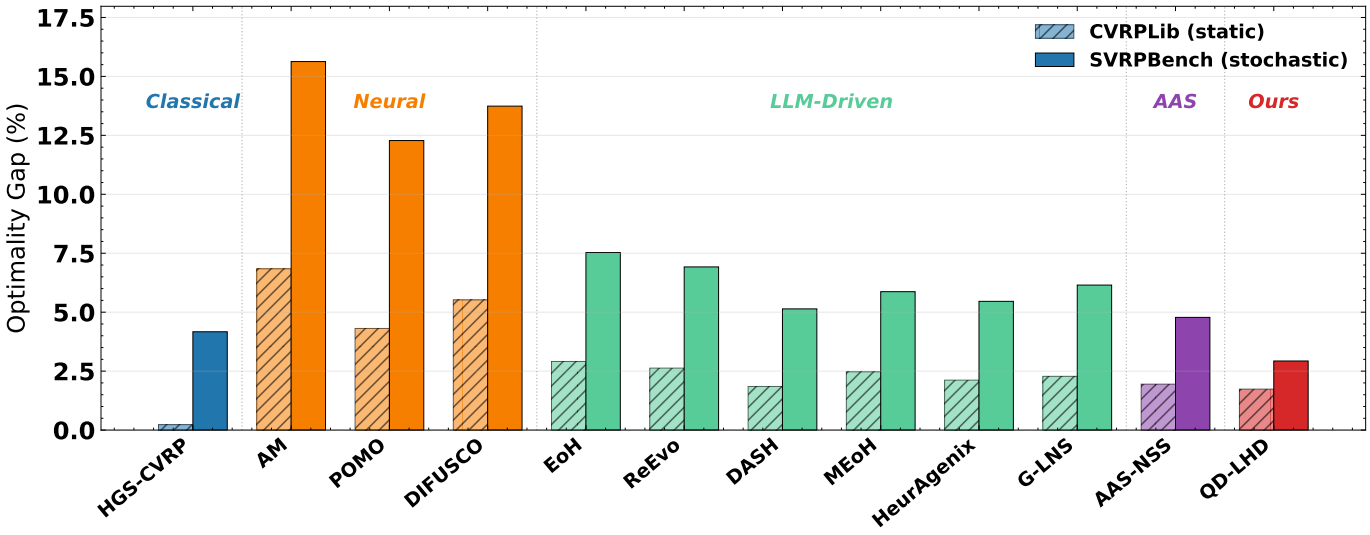


Fig. 3. Optimality gap (%) on static CVRPLib (hatched) versus stochastic SVRPBench (solid) for each method, grouped by tier. QD-LHD exhibits the smallest absolute gap on SVRPBench and the smallest relative degradation from static to stochastic conditions.

TABLE VI
RESULTS ON TSP BENCHMARKS. NOTATION FOLLOWS TABLE IV.

		TSPLib		Perturbed TSP ($\sigma = 0.05$)		
	Method	Gap	Time	Gap	Time	CV
Cls.	LKH-3	**0.02**	15.3	2.86†	16.1	0.15†
Neural	AM	5.76†	**0.6**	11.42†	**0.7**	0.31†
	POMO	3.48†	0.8	8.73†	0.9	0.27†
	DIFUSCO	4.21†	2.9	9.85†	3.1	0.29†
LLM-Driven	EoH	2.37†	0.9	5.94†	1.0	0.17†
	ReEvo	2.14†	1.1	5.51†	1.2	0.16†
	DASH	1.52†	1.4	4.18†	1.5	0.12†
	MEoH	1.89†	1.2	4.72†	1.3	0.14†
	HeurAgenix	1.68†	0.8	4.43†	0.9	0.13†
	G-LNS	1.93†	1.6	5.07†	1.7	0.15†
AAS	AAS-NSS [49]	1.58†	1.1	3.84†	1.2	0.11†
Ours	QD-LHD	1.41	1.0	**2.31**	1.1	**0.06**

TABLE VII
PERTURBED-TSP SENSITIVITY TO THE NOISE AMPLITUDE σ (100 INSTANCES, 30 RUNS EACH). BEST PER ROW GROUP IN **BOLD**.

	$\sigma = 0.05$		$\sigma = 0.10$	
Method	Gap (%)	CV	Gap (%)	CV
LKH-3	2.86	0.15	4.94	0.22
DASH	4.18	0.12	6.31	0.17
HeurAgenix	4.43	0.13	6.72	0.18
QD-LHD	**2.31**	**0.06**	**3.52**	**0.09**

shifts in input geometry. LLM-driven baselines show tighter distributions, yet DASH still has a broad upper tail. QD-LHD produces the tightest box, and the archive's selector suppresses variance by matching each perturbed instance to a heuristic whose behavioral profile aligns with the shifted geometry.

LKH-3, although strongest on static instances, also degrades under node perturbation and has a wider distribution than QD-LHD. The classical-solver protocol here runs LKH-3 independently on each of the $R = 10$ realizations with Helsgaun's default parameters [45] and aggregates the results, with no state shared between realizations; the reported degradation reflects the variance of the realization sample under that protocol. Two factors explain why a solver of this maturity leaves a 2.86% gap here. Each realization's reference cost is the best tour found by prolonged multi-restart runs, whereas the table entry comes from one default-parameter run per realization within the deployment budget, and as a result the reported gap compares a budget-limited run against the strongest tour the same solver can reach. Noise at $\sigma = 0.05\times$ the bounding-box diagonal, in addition, exceeds the typical nearest-neighbor spacing of TSPLib instances; the perturbed realizations lose the clustered geometry for which LKH-3's default candidate-set parameters were calibrated, and every re-solve starts cold. The wider spread across realizations in the CV column follows directly.

3) Sensitivity to Perturbation Strength: We also evaluate the perturbed-TSP benchmark at a stronger noise amplitude, $\sigma = 0.10\times$ bounding box diagonal—twice the default. Table VII reports the four strongest methods at both amplitudes. All methods degrade, but the ordering is preserved and QD-LHD's increase is smaller than that of LKH-3 or DASH. QD-LHD's coefficient of variation remains roughly half that of the next-best method at each amplitude. The archive's advantage therefore scales with the noise: as the test distribution drifts further from training, cell-local specialization absorbs more of the error than a single-heuristic baseline can.

4) Archive Metrics: The TSP archive has slightly lower coverage than the CVRP archive, which we attribute to narrower structural variation among TSP heuristics: the constraint-free nature of the TSP limits the diversity of algorithmic strategies that the LLM can produce compared to routing with capacity and time window constraints. Its average complexity and code length are also lower, consistent with the simpler problem structure. The shift degradation ratio remains comparable to CVRP and well below DASH's ratio.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

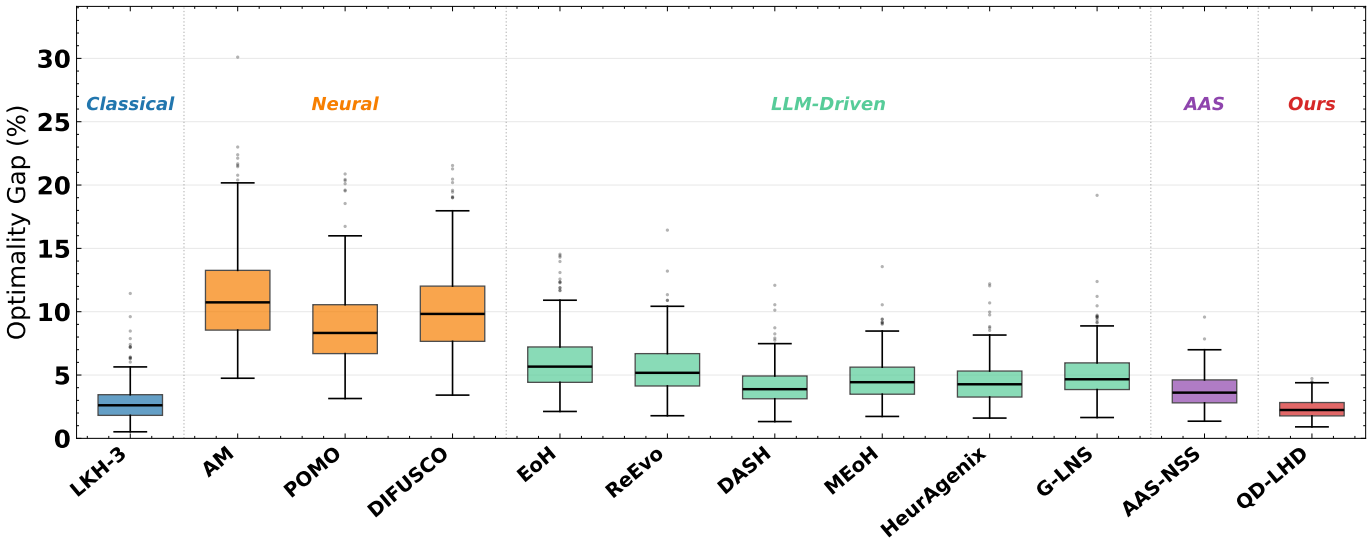


Fig. 4. Box plots of the optimality gap (%) on perturbed-TSP instances ($\sigma = 0.05$, 100 instances, 30 runs each) for all methods. QD-LHD achieves both the lowest median and the narrowest spread, which indicates that the archive-based selector consistently retrieves well-matched heuristics regardless of the specific perturbation pattern.

TABLE VIII
RESULTS ON JSSP BENCHMARKS. NOTATION FOLLOWS TABLE IV.

		Taillard		Dynamic JSSP		
	Method	Gap	Time	Gap	Time	CV
Cls.	CP-SAT	**0.41**	62.7	$5.63^{\dagger}$	68.4	$0.22^{\dagger}$
Neural	AM	$7.93^{\dagger}$	**0.5**	$14.17^{\dagger}$	**0.6**	$0.34^{\dagger}$
	POMO	$5.82^{\dagger}$	0.7	$10.94^{\dagger}$	0.8	$0.29^{\dagger}$
	DIFUSCO	$6.54^{\dagger}$	2.4	$12.36^{\dagger}$	2.6	$0.31^{\dagger}$
LLM-Driven	EoH	$3.71^{\dagger}$	0.8	$7.82^{\dagger}$	0.9	$0.19^{\dagger}$
	ReEvo	$3.42^{\dagger}$	1.0	$7.21^{\dagger}$	1.1	$0.17^{\dagger}$
	DASH	$2.36^{\dagger}$	1.3	$5.47^{\dagger}$	1.4	$0.13^{\dagger}$
	MEoH	$2.85^{\dagger}$	1.1	$6.13^{\dagger}$	1.2	$0.15^{\dagger}$
	HeurAgenix	$2.57^{\dagger}$	0.7	$5.74^{\dagger}$	0.8	$0.14^{\dagger}$
	G-LNS	$3.04^{\dagger}$	1.5	$6.58^{\dagger}$	1.6	$0.16^{\dagger}$
AAS	AAS-NSS [49]	$2.42^{\dagger}$	1.0	$5.06^{\dagger}$	1.1	$0.12^{\dagger}$
Ours	QD-LHD	2.18	0.9	**3.15**	1.0	**0.07**

D. Results on Job Shop Scheduling

Table VIII presents the results on the JSSP benchmarks. Scheduling differs from routing in that the decision variables are temporal orderings of operations on machines, and the dynamic variant introduces random job arrivals during execution. This section examines whether the archive-based approach transfers to a structurally different problem family.

1) Static Performance: CP-SAT achieves the best static gap by solving a constraint programming model to near-optimality, though it requires over a minute per instance on the larger configurations (50×20, 100×20). Among learning-based methods, QD-LHD obtains the best gap and DASH is closest. The margins are consistent with those observed on routing and TSP: QD-LHD's archive provides a slight but statistically reliable edge on static instances ($p < 0.05$ against all Tier 3 baselines), while the classical solver remains out of reach in pure solution quality.

2) Dynamic Performance: The dynamic JSSP variant, where jobs arrive according to a Poisson process and the schedule must be updated online, produces the largest inter-tier separations of any benchmark. CP-SAT degrades because its constraint model must be re-solved after each arrival, and the time budget per re-solve is tight. Neural methods follow similar downward trajectories.

QD-LHD has the lowest gap and coefficient of variation across all methods. The archive contains cells specialized for different arrival intensities and shop configurations: heuristics in cells corresponding to high-arrival-rate features tend to use simpler priority dispatching rules that can be applied quickly, while those in low-arrival cells employ more deliberate local search. The online selector routes each dynamic instance to the appropriate cell based on the observed arrival rate and shop utilization at decision time.

3) Multi-Metric Comparison: Fig. 5 presents a radar chart across five normalized metrics—static quality, dynamic quality, robustness (CV), speed, and shift resilience—for the five most competitive methods, each axis normalized to $[0, 1]$ with 1 representing the best score.

QD-LHD achieves the largest enclosed area, with the highest scores on dynamic quality, robustness, and shift resilience. Its speed is comparable to DASH and HeurAgenix, since the deployment cost is dominated by heuristic execution rather than archive lookup. CP-SAT dominates on static quality but collapses on speed and shift resilience, while POMO is fast but weak on both quality axes. DASH and HeurAgenix occupy a middle ground, with balanced profiles that are consistently enclosed by QD-LHD's polygon.

This multi-metric view reinforces a pattern observed across all three problem families: no single-algorithm method matches QD-LHD's simultaneous strength on quality, robustness, and shift resilience.

4) Archive Metrics: The JSSP archive coverage lies between the CVRP and TSP values: the scheduling domain

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

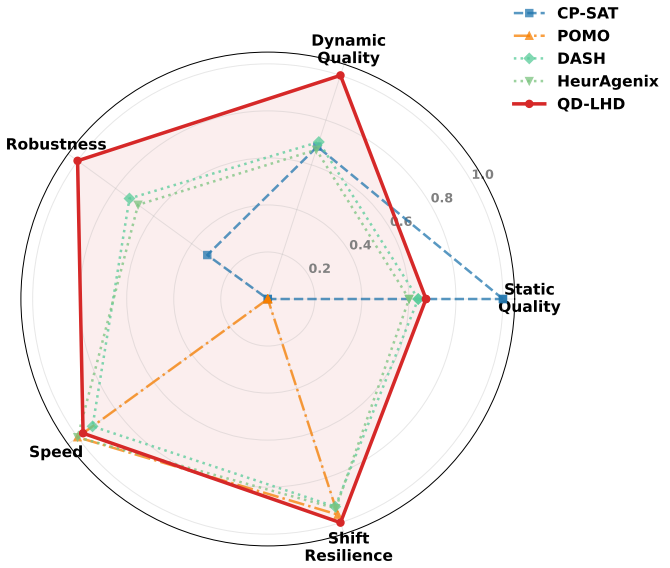


Fig. 5. Radar chart comparing five representative methods on the dynamic JSSP across five normalized metrics (1 = best). QD-LHD (red, solid) encloses the largest area, which reflects its balanced superiority across quality, robustness, speed, and shift resilience. CP-SAT excels on static quality but collapses on speed and shift resilience. POMO is fast but weak on all quality axes.

offers richer structural variation than the TSP, due to machine assignment choices and diverse dispatching rules, but less than CVRP, which combines routing, packing, and time window decisions. Its average complexity and code length are also intermediate. The shift degradation ratio is the lowest across all three problem families. We attribute this to the temporal nature of dynamic JSSP: arrival-rate variation maps cleanly to distinct behavioral niches in the archive, and the selector can exploit this structure effectively.

E. Ablation Study and Sensitivity Analysis

Three ablation variants isolate the contribution of each architectural component. *QD-LHD-SE* replaces the MAP-Elites archive with a standard $(\mu + \lambda)$ evolutionary population of size $\mu = 50$ and produces, in effect, a SHAP-guided single-population LHD method. *QD-LHD-BM* retains the archive but omits SHAP attributions from the mutation prompt; the LLM receives only the parent code and its performance vector. *QD-LHD-HD* preserves SHAP mutation guidance yet replaces data-driven behavioral descriptors with three hand-designed instance features: problem scale, constraint tightness, and demand variability. Table IX reports results on SVRPBench and dynamic JSSP, the two benchmarks where performance differences are most pronounced.

1) Effect of the Archive Structure: The largest degradation occurs in QD-LHD-SE. Its SVRPBench optimality gap rises to 4.87%—up from 2.93% in the full system—and its coefficient of variation jumps to 0.15, on par with DASH in Table IV. Since all test instances are served by the single evolved heuristic, stochastic performance depends entirely on whatever incidental robustness that one algorithm possesses. The archive, in other words, is the primary source of distributional

TABLE IX
ABLATION RESULTS AND SCALARIZATION-WEIGHT SENSITIVITY ON SVRPBENCH (STOCHASTIC CVRP) AND DYNAMIC JSSP. COV. DENOTES ARCHIVE COVERAGE (%); N/A APPLIES TO THE VARIANT WITHOUT AN ARCHIVE. DEFAULT WEIGHTS ARE $(w_1, w_2, w_3) = (0.6, 0.2, 0.2)$. BEST IN **BOLD**.

	SVRPBench				Dynamic JSSP	
Variant	Gap	CV	Cov.	QD	Gap	CV
QD-LHD-SE (no archive)	4.87	0.15	N/A	N/A	5.28	0.13
QD-LHD-BM (blind mutation)	3.84	0.11	64.3	521.6	4.12	0.10
QD-LHD-HD (hand-designed desc.)	3.31	0.10	70.5	589.2	3.68	0.09
QD-LHD (full, default w)	**2.93**	0.08	**73.2**	**614.8**	**3.15**	0.07
$w = (0.2, 0.6, 0.2)$ time-heavy	3.41	0.09	72.8	609.1	3.62	0.08
$w = (0.2, 0.2, 0.6)$ rob-heavy	3.18	**0.07**	73.0	611.6	3.44	**0.06**
$w = (0.34, 0.33, 0.33)$ balanced	3.04	0.08	73.1	612.9	3.28	0.07

resilience, not the SHAP feedback mechanism in isolation. Without it, the framework degrades to a well-guided but distribution-specific heuristic designer.

2) Effect of SHAP Mutation Guidance: QD-LHD-BM retains the archive structure and thus preserves more robustness than QD-LHD-SE; however, archive coverage falls from 73.2% to 64.3%, and the optimality gaps reach 3.84% and 4.12% on SVRPBench and dynamic JSSP respectively. Without structural feedback, the LLM proposes mutations that amount to undirected exploration: the archive fills, but the elites in each cell tend to be of lower quality and the coverage plateau is reached later. SHAP guidance, in this sense, accelerates the evolutionary search—mutations target structurally productive changes rather than random code perturbations.

3) Effect of Descriptor Type: The hand-designed descriptor variant reduces the SVRPBench gap modestly to 3.31% and achieves 70.5% coverage, close to the full system's 73.2%. The deterioration relative to QD-LHD is concentrated on stochastic and dynamic benchmarks. Instance-level features such as scale and constraint tightness capture coarser problem properties than SHAP-derived code-structural features. Two heuristics that differ in loop depth or recursion structure may share identical instance-feature coordinates under the hand-designed scheme, competing for the same cell despite distinct behavioral profiles. SHAP-derived descriptors place structurally different heuristics in separate cells regardless of instance similarity.

4) Sensitivity to Scalarization Weights: The scalarization weights in (8) trade the three objectives off inside each cell without changing the descriptor axes that organize the archive. Table IX reports the three alternative settings alongside the default. Shifting weight from quality to runtime ($w = (0.2, 0.6, 0.2)$, time-heavy) lowers measured runtime at a cost of 0.48 gap points; shifting weight to robustness ($w = (0.2, 0.2, 0.6)$) lowers the coefficient of variation by 0.01 and gives up 0.25 gap points. The balanced setting sits between the two. Archive coverage stays within 0.4 percentage points and the QD-score within 6 units across all four configurations, so the archive structure is decoupled from the scalarization: the weight choice acts on per-cell selection after the behavioral space has already been illuminated.

5) Sensitivity to Archive Granularity: Table X examines how the number of behavioral dimensions d and bins B affect performance at a fixed budget of 500 generations.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

TABLE X
SENSITIVITY TO ARCHIVE DIMENSIONS d AND BINS B ON SVRPBENCH (500 GENERATIONS, 30 RUNS). COST IS REPORTED PER INDEPENDENT ARCHIVE-CONSTRUCTION RUN. DEFAULT CONFIGURATION IN BOLD. $*$: BUDGET EXHAUSTED BEFORE COVERAGE PLATEAU.

d	B	K	Cov. (%)	QD-Score	Gap (%)	Cost ($)
2	10	100	82.1	623.4	3.24	34
3	5	125	91.4	598.3	3.47	31
3	**10**	**1,000**	**73.2**	**614.8**	**2.93**	**85**
3	15	3,375	42.6*	511.7	3.71	85*
4	10	10,000	23.8*	398.4	4.53	85*

TABLE XI
LLM BACKBONE COMPARISON ON SVRPBENCH ($d = 3$, $B = 10$, 500 GENERATIONS, 30 RUNS). COST IS REPORTED PER INDEPENDENT ARCHIVE-CONSTRUCTION RUN. BEST IN **BOLD**.

LLM	Gap (%)	CV	Valid (%)	Cost ($)
GPT-5.4	**2.93**	**0.08**	**91.3**	85
Claude Opus 4.6	3.07	0.09	89.7	78
Qwen3-235B-A22B	3.42	0.11	84.2	**23**

The relationship is non-monotone. With $d = 2, B = 10$ ($K = 100$ cells), coverage reaches 82.1%, yet the archive cannot represent sufficient heuristic diversity: 100 behavioral niches prove too coarse to distinguish the range of algorithmic strategies the LLM produces, and the SVRPBench optimality gap rises to 3.24%. At the other extreme, $d = 3, B = 15$ ($K = 3{,}375$) leaves 57.4% of cells vacant when the generation budget is exhausted, and the gap rises to 3.71%. This pattern reflects Proposition 4.1: for a fixed budget, coverage decreases roughly exponentially with the number of cells. The default configuration ($d = 3, B = 10, K = 1{,}000$) lies at a favorable point on this trade-off. An increase to $d = 4$ ($K = 10{,}000$) compounds the problem further, and coverage falls to 23.8% despite consuming the full generation budget.

The $d = 3, B = 5$ configuration achieves the highest coverage (91.4%) but falls short on optimality gap (3.47%), since the coarser grid conflates heuristics from structurally different instance types. Coverage alone is thus an insufficient proxy for archive quality; the QD-score identifies the default as the most balanced option.

6) Sensitivity to LLM Backbone: Table XI compares three LLM backbones on SVRPBench under the default archive configuration. GPT-5.4 achieves the lowest optimality gap (2.93%) and the highest syntactically valid code rate (91.3%). Claude Opus 4.6 is closely competitive: its gap of 3.07% differs by 0.14 percentage points, and its lower cost ($78 versus $85) reflects a more compact token distribution on code-generation tasks. Qwen3-235B-A22B, an open-source alternative, widens the gap to 3.42% with a valid code rate of 84.2%, yet it reduces the API cost to $23—a 3.7-fold reduction relative to GPT-5.4.

The framework is not tightly coupled to any particular backbone: once a model produces valid code at a reasonable rate, archive quality depends on both call volume and per-call capability. A compact backbone-selection protocol follows—benchmark each candidate on ~ 20 canonical seed prompts, record the valid-code rate and optimality gap, and pick the cheapest that clears a pre-set valid-rate threshold. Qwen3-235B-A22B already clears such a threshold at $3.7\times$ lower cost than GPT-5.4 (Table XI).

7) Archive Convergence and Descriptor Stability: Fig. 6 traces the archive evolution over 500 generations. Coverage follows a saturating trajectory across all three problem domains: the archive fills rapidly during the first 150 generations as the LLM explores diverse algorithmic strategies, then growth decelerates as the remaining empty cells correspond to increasingly specialized behavioral niches. CVRP reaches the highest final coverage (73.2%) due to its richer design space, while TSP plateaus at 68.7%. The QD-score curves mirror this pattern with a slight lag, since cells are first occupied by mediocre heuristics that are later replaced by higher-quality elites through the cell competition mechanism.

Panel (c) of Fig. 6 shows which AST features are selected as the top-3 behavioral dimensions at each surrogate retraining point ($\Delta = 50$ generations). During the first three retraining cycles (generations 50–150), the ranking is unstable: `ast_depth` and `num_arith` appear transiently as the surrogate has insufficient training data. After generation 200—corresponding to approximately 250 evaluated heuristics—the top-3 stabilizes at `loop_nesting`, `num_branches`, and `has_sorting`, which remain fixed through the remaining seven retraining cycles. This empirical observation is consistent with Proposition 4.2: descriptor drift is controlled only after the local fitted-tree structure stabilizes.

VI. CONCLUSION

LLM-driven heuristic design typically converges toward a single best algorithm—or a small Pareto front—tuned to one instance distribution; under shift, the process must restart at non-trivial cost and the resulting algorithms carry no robustness guarantee. QD-LHD reframes the problem as quality-diversity search: the evolutionary loop fills a structured archive of specialized heuristics indexed by code-structural descriptors derived from SHAP analysis of a surrogate, and at deployment the best-matched heuristic for each new instance is retrieved in $O(K)$ time without further LLM queries.

Three observations follow from analysis and experiment together. The first concerns the archive itself. Proposition 4.3 establishes that, on selector-covered regions, the expected performance under distributional shift is bounded above by $\sum_k \pi_k[\mathbb{E}_{\mathcal{P}_k}[f_{\mathrm{qual}}(h_k^*)] + L_k\, W_1(\mathcal{P}_k, \mathcal{Q}_k)] + \varepsilon_{\mathrm{sel}} M$. The archive decomposes the bound over cells: each local Lipschitz constant L_k is controlled on a region where the specialized heuristic remains feasible, while $\varepsilon_{\mathrm{sel}}$ captures the deployment mass not covered by the selected heuristic's feasible support. Single-heuristic methods have no analogous decomposition: their performance under shift depends on a single global Lipschitz constant that must absorb feasibility-boundary transitions where no finite constant exists. The ablation confirms the separation: removing the archive (QD-LHD-SE) raises the SVRPBench gap from 2.93% to 4.87% and the CV from 0.08 to 0.15, putting performance in the range of DASH and HeurAgenix under identical SHAP mutation guidance.

The second observation concerns behavioral descriptors. Heuristic programs have no canonical behavioral space:

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

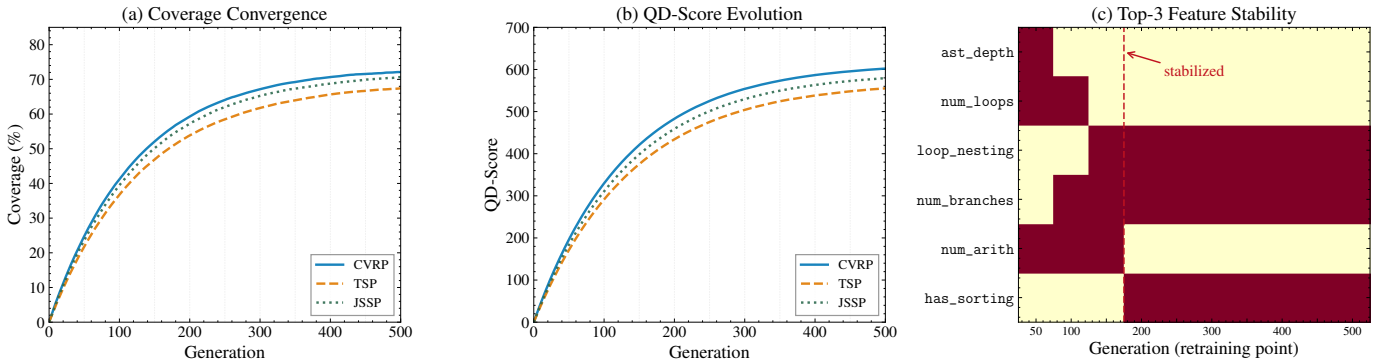


Fig. 6. Archive evolution over 500 generations. (a) Coverage converges in a saturating trajectory, with CVRP reaching the highest final value due to its richer algorithmic design space. (b) QD-score rises with a slight lag behind coverage as cell competition replaces initial elites with higher-quality heuristics. (c) Top-3 behavioral descriptor features at each surrogate retraining point (every 50 generations). The descriptor axes stabilize after generation 200 (dashed red line), consistent with Proposition 4.2. Vertical gray dotted lines mark surrogate retraining events.

robotic QD uses limb-contact patterns and game design uses level statistics, but no physical interpretation exists for algorithmic code. SHAP analysis treats the behavioral space as a derived quantity—the AST features most predictive of performance variation become the axes that organize diversity. Proposition 4.2 bounds descriptor drift only under local stability of the fitted tree ensemble—and the experiments confirm stabilization after 200–300 evaluated heuristics, within the 500-generation budget. The property is not specific to heuristic design; domains where behavioral axes are unclear a priori—prompt optimization, architecture search, operator selection—may benefit from a similar SHAP-derived construction.

The third observation concerns the LLM as a variation operator. Blind mutation (QD-LHD-BM) reduces coverage from 73.2% to 64.3% and raises the SVRPBench gap from 2.93% to 3.84%: without SHAP feedback, offspring cluster near the parent and underexplored cells receive fewer visits, not because valid offspring are scarcer but because mutations lack directional guidance. With feedback, the prompt identifies productive versus detrimental code-structural changes and the LLM proposes targeted modifications; the operator reasons about structure, not just fitness. The backbone comparison shows this holds across commercial and open-source models, at different efficiency levels.

Several constraints bound the present work. The surrogate warm-up period of 200–300 evaluations leaves early-generation placements on raw AST features; later retraining partially but not fully corrects premature specialization. The $K = B^d$ cell count grows exponentially in d, and Proposition 4.1 implies rapid coverage deterioration beyond $d = 3$ at a fixed budget—a limit the sensitivity experiments confirm. The reachability probability $p_{\min}$ and the cell-local Lipschitz constants L_k in the analytical bounds are not computable in closed form; the bounds hold in principle but cannot be evaluated precisely for a given problem domain. The instance batch I must be representative of the deployment distribution; a biased batch produces uneven coverage the selector cannot compensate for. Any LLM-driven heuristic-generation pipeline also inherits the dual-use exposure documented for LLM code generation more broadly—Gu et al. [51] show that optimization-algorithm requests can be rewritten to bypass safety alignment—so production deployments should pair the sandbox isolation of Section V-A with code-level safety filters on the generated code.

Several directions extend the framework naturally. CVT-MAP-Elites [28] may push feasible behavioral dimensionality beyond $d = 3$ within acceptable LLM budgets. An online archive update mechanism—where deployment performance feeds back to trigger targeted generation on underperforming cells—would extend the current zero-shot retrieval with continual adaptation, trading the audit-friendly fixed library for responsiveness to distributional drift; the two modes share the same archive representation, so the choice is a deployment-time switch rather than a framework change. Cross-family transfer is unexplored: an archive built on routing may warm-start scheduling evolution by seeding with structurally compatible elites, since code patterns such as priority queues, greedy insertion, and 2-opt recur across domains. Adapting descriptor axes to the current Pareto trade-off surface, rather than raw performance prediction, may improve coverage efficiency under conflicting objectives.

Taken together, LLMs do not merely accelerate the search for good algorithms; they make it tractable to maintain a structured space of algorithms. Quality-diversity search provides the population-management principle that turns this capacity into a deployable heuristic library—a combination that neither single- nor multi-objective heuristic evolution has captured. Whether it extends further into automated algorithm configuration, neural architecture search, or per-instance algorithm selection [37], [38] remains an open question, and the analytical connection to distributional robustness gives a basis for pursuing it.

Code Availability

The code of QD-LHD is available at https://anonymous.4open.science/status/LLM_CO-DD67.

References

[1] K. Sörensen, "Metaheuristics—the metaphor exposed," *International Transactions in Operational Research*, vol. 22, no. 1, pp. 3–18, 2015.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

[2] A. E. Eiben and S. K. Smit, "Parameter tuning for configuring and analyzing evolutionary algorithms," *Swarm and evolutionary computation*, vol. 1, no. 1, pp. 19–31, 2011.

[3] N. Pillay and R. Qu, *Hyper-heuristics: theory and applications*. Springer, 2018.

[4] Y. Jin, H. Wang, T. Chugh, D. Guo, and K. Miettinen, "Data-driven evolutionary optimization: An overview and case studies," *IEEE Transactions on Evolutionary Computation*, vol. 23, no. 3, pp. 442–458, 2018.

[5] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi *et al.*, "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, no. 7995, pp. 468–475, 2024.

[6] F. Liu, X. Tong, M. Yuan, X. Lin, F. Luo, Z. Wang, Z. Lu, and Q. Zhang, "Evolution of heuristics: Towards efficient automatic algorithm design using large language model," *arXiv preprint arXiv:2401.02051*, 2024.

[7] H. Ye, J. Wang, Z. Cao, F. Berto, C. Hua, H. Kim, J. Park, and G. Song, "Reevo: Large language models as hyper-heuristics with reflective evolution," *Advances in neural information processing systems*, vol. 37, pp. 43 571–43 608, 2024.

[8] N. Van Stein and T. Bäck, "Llamea: A large language model evolutionary algorithm for automatically generating metaheuristics," *IEEE Transactions on Evolutionary Computation*, vol. 29, no. 2, pp. 331–345, 2024.

[9] R. Wang, Y. Huang, M. Li, J. Li, D. Liang, B. Simons, P. Ke, S. Liang, and K. Qin, "Rethinking llm-driven heuristic design: Generating efficient and specialized solvers via dynamics-aware optimization," *arXiv preprint arXiv:2601.20868*, 2026.

[10] S. Yao, F. Liu, X. Lin, Z. Lu, Z. Wang, and Q. Zhang, "Multi-objective evolution of heuristic using large language model," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 25, 2025, pp. 27 144–27 152.

[11] A. Hottung, F. Berto, C. Hua, N. G. Zepeda, D. Wetzel, M. Römer, H. Ye, D. Zago, M. Poli, S. Massaroli *et al.*, "Vrpagent: Llm-driven discovery of heuristic operators for vehicle routing problems," *arXiv preprint arXiv:2510.07073*, 2025.

[12] X. Yang, L. Zhang, H. Qian, L. Song, and J. Bian, "Heuragenix: Leveraging llms for solving complex combinatorial optimization challenges," *arXiv preprint arXiv:2506.15196*, 2025.

[13] B. Zhao, H. Wang, and L. Zeng, "G-lns: Generative large neighborhood search for llm-based automatic heuristic design," *arXiv preprint arXiv:2602.08253*, 2026.

[14] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[15] N. van Stein, A. V. Kononova, L. Kotthoff, and T. Bäck, "Llamea-sage: Guiding automated algorithm design with structural feedback from explainable ai," *arXiv preprint arXiv:2601.21511*, 2026.

[16] W. Kool, H. Van Hoof, and M. Welling, "Attention, learn to solve routing problems!" *arXiv preprint arXiv:1803.08475*, 2018.

[17] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, and S. Min, "POMO: Policy optimization with multiple optima for reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 21 188–21 198.

[18] A. Heakl, Y. S. Shaaban, M. Takác, S. Lahlou, and Z. Iklassov, "Svrp-bench: A realistic benchmark for stochastic vehicle routing problem," *arXiv preprint arXiv:2505.21887*, 2025.

[19] J.-B. Mouret and J. Clune, "Illuminating search spaces by mapping elites," *arXiv preprint arXiv:1504.04909*, 2015.

[20] A. Cully and Y. Demiris, "Quality and diversity optimization: A unifying modular framework," *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 2, pp. 245–259, 2017.

[21] E. K. Burke, M. Gendreau, M. Hyde, G. Kendall, G. Ochoa, E. Özcan, and R. Qu, "Hyper-heuristics: A survey of the state of the art," *Journal of the Operational Research Society*, vol. 64, no. 12, pp. 1695–1724, 2013.

[22] F. Liu, Y. Yao, P. Guo, Z. Yang, X. Lin, Z. Zhao, X. Tong, K. Mao, Z. Lu, Z. Wang *et al.*, "A systematic survey on large language models for algorithm design," *ACM Computing Surveys*, vol. 58, no. 8, pp. 1–32, 2026.

[23] Z. Ma, H. Guo, Y.-J. Gong, J. Zhang, and K. C. Tan, "Toward automated algorithm design: A survey and practical guide to meta-black-box-optimization," *IEEE Transactions on Evolutionary Computation*, 2025.

[24] H. Guo, Z. Ma, J. Chen, Y. Ma, Z. Cao, X. Zhang, and Y.-J. Gong, "ConfigX: Modular configuration for evolutionary algorithms via multitask reinforcement learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, 2025.

[25] H. Guo, Z. Ma, Y. Ma, X. Zhang, J. Chen, and Y.-J. Gong, "DesignX: Human-competitive algorithm designer for black-box optimization," in *Advances in Neural Information Processing Systems*, vol. 38, 2025.

[26] Z. Ma, H. Guo, Z. Jiang, Z. Cao, and Y.-J. Gong, "Meta-black-box-optimization through offline q-function learning with mamba architecture," 2025.

[27] S. Zhang, S. Liu, N. Lu, J. Wu, J. Liu, Y.-S. Ong, and K. Tang, "Llm-driven instance-specific heuristic generation and selection," *arXiv preprint arXiv:2506.00490*, 2025.

[28] V. Vassiliades, K. Chatzilygeroudis, and J.-B. Mouret, "Using centroidal voronoi tessellations to scale up the multidimensional archive of phenotypic elites algorithm," *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 4, pp. 623–630, 2017.

[29] M. C. Fontaine, J. Togelius, S. Nikolaidis, and A. K. Hoover, "Covariance matrix adaptation for the rapid illumination of behavior space," in *Proceedings of the 2020 genetic and evolutionary computation conference*, 2020, pp. 94–102.

[30] A. Cully, J. Clune, D. Tarapore, and J.-B. Mouret, "Robots that can adapt like animals," *Nature*, vol. 521, no. 7553, pp. 503–507, 2015.

[31] S. Kuroki, T. Nakamura, T. Akiba, and Y. Tang, "Agent skill acquisition for large language models via cycleqd," *arXiv preprint arXiv:2410.14735*, 2024.

[32] O. Mersmann, B. Bischl, H. Trautmann, M. Preuss, C. Weihs, and G. Rudolph, "Exploratory landscape analysis," in *Proceedings of the 13th annual conference on Genetic and evolutionary computation*, 2011, pp. 829–836.

[33] P. Kerschke and H. Trautmann, "Comprehensive feature-based landscape analysis of continuous and constrained optimization problems using the R-package flacco," in *Applications in Statistical Computing*. Springer, 2019, pp. 93–123.

[34] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.

[35] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, R. Katz, J. Himmelfarb, N. Banerjee, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.

[36] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.

[37] J. R. Rice, "The algorithm selection problem," in *Advances in Computers*. Elsevier, 1976, vol. 15, pp. 65–118.

[38] M. Lindauer, H. H. Hoos, F. Hutter, and T. Schaub, "AutoFolio: An automatically configured algorithm selector," *Journal of Artificial Intelligence Research*, vol. 53, pp. 745–778, 2015.

[39] C. Villani, *Optimal transport: old and new*. Springer, 2009, vol. 338.

[40] E. Uchoa, D. Pecin, A. Pessoa, M. Poggi, T. Vidal, and A. Subramanian, "New benchmark instances for the capacitated vehicle routing problem," *European Journal of Operational Research*, vol. 257, no. 3, pp. 845–858, 2017.

[41] M. M. Solomon, "Algorithms for the vehicle routing and scheduling problems with time window constraints," *Operations Research*, vol. 35, no. 2, pp. 254–265, 1987.

[42] G. Reinelt, "TSPLIB—a traveling salesman problem library," *ORSA Journal on Computing*, vol. 3, no. 4, pp. 376–384, 1991.

[43] É. Taillard, "Benchmarks for basic scheduling problems," *European Journal of Operational Research*, vol. 64, no. 2, pp. 278–285, 1993.

[44] D. Bertsimas and M. Sim, "Robust discrete optimization and network flows," *Mathematical Programming*, vol. 98, no. 1–3, pp. 49–71, 2003.

[45] K. Helsgaun, "An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems," *Roskilde: Roskilde University*, vol. 12, pp. 966–980, 2017.

[46] L. Perron, F. Didier, and S. Gay, "The cp-sat-lp solver (invited talk)," in *29th International Conference on Principles and Practice of Constraint Programming (CP 2023)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2023, pp. 3–1.

[47] T. Vidal, "Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood," *Computers & Operations Research*, vol. 140, p. 105643, 2022.

[48] Z. Sun and Y. Yang, "Difusco: Graph-based diffusion solvers for combinatorial optimization," *Advances in neural information processing systems*, vol. 36, pp. 3706–3731, 2023.

[49] C. Gao, H. Shang, K. Xue, and C. Qian, "Neural solver selection for combinatorial optimization," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, Vancouver, Canada, 2025.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Evolutionary Computation. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TEVC.2026.3721299

[50] A. Vargha and H. D. Delaney, "A critique and improvement of the cl common language effect size statistics of mcgraw and wong," *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000.

[51] H. Gu, H. Wang, Y. Mei, M. Zhang, and Y. Jin, "Overlooked safety vulnerability in llms: Malicious intelligent optimization algorithm request and its jailbreak," *arXiv preprint arXiv:2601.00213*, 2026.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 29,2026 at 12:09:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.